CDC

A. APARTADO A

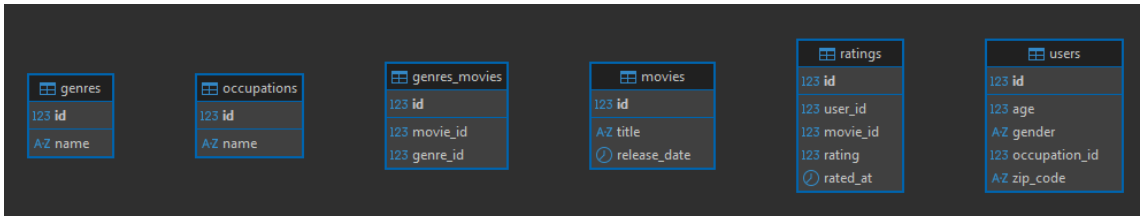
1) En el servidor MySql crea una nueva base de datos llamada movielens y dentro de ella ejecuta el script del punto anterior para generar las tablas y sus datos. (Investiga con que comando has de lanzar el script)

```
BEGIN;

DROP DATABASE IF EXISTS movielens;

CREATE DATABASE movielens;

USE movielens;

DROP TABLE IF EXISTS occupations;
CREATE TABLE occupations (
  id integer NOT NULL,
  name varchar(255),
  PRIMARY KEY (id)
);
INSERT INTO occupations VALUES (1,'Administrator'),(2,'Artist'),(3,'Doctor'),(4,'Educator'),(5,'Engineer'),(6,'Entertainm

DROP TABLE IF EXISTS users;
CREATE TABLE users (
  id integer NOT NULL,
  age integer,
  gender char(1),
  occupation_id integer,
  zip_code varchar(255),
  PRIMARY KEY (id)
);
INSERT INTO users VALUES (1,24,'M',20,'85711'),(2,53,'F',14,'94043'),(3,23,'M',21,'32067'),(4,24,'M',20,'43537'),(5,33,'F

DROP TABLE IF EXISTS ratings;
CREATE TABLE ratings (
  id integer NOT NULL,
  user_id integer,
```

2) Puede aparecer un error al cargar la tabla movies ¿Puedes detectar de qué se trata?

   No me ha salido, pero entiendo que pueda deberse a la codificación de los caracteres de los datos de la base de datos.

3) Muestra las tablas generadas, el contenido y estructura de las tablas movies, ocupations y users.



Ejemplo de datos

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grilla | Texto

	id	title	release_date
1	1	Toy Story (1995)	1995-01-01
2	2	GoldenEye (1995)	1995-01-01
3	3	Four Rooms (1995)	1995-01-01
4	4	Get Shorty (1995)	1995-01-01
5	5	Copycat (1995)	1995-01-01
6	6	Shanghai Triad (Yao a yao yao dao waipo qiao) (1995)	1995-01-01
7	7	Twelve Monkeys (1995)	1995-01-01
8	8	Babe (1995)	1995-01-01
9	9	Dead Man Walking (1995)	1995-01-01
10	10	Richard III (1995)	1996-01-22
11	11	Seven (Se7en) (1995)	1995-01-01
12	12	Usual Suspects, The (1995)	1995-08-14
13	13	Mighty Aphrodite (1995)	1995-10-30
14	14	Postino, Il (1994)	1994-01-01
15	15	Mr. Holland's Opus (1995)	1996-01-29
16	16	French Twist (Gazon maudit) (1995)	1995-01-01
17	17	From Dusk Till Dawn (1996)	1996-02-05
18	18	White Balloon, The (1995)	1995-01-01
19	19	Antonia's Line (1995)	1995-01-01
20	20	Angels and Insects (1995)	1995-01-01
21	21	Muppet Treasure Island (1996)	1996-02-16
22	22	Braveheart (1995)	1996-02-16
23	23	Taxi Driver (1976)	1996-02-16
24	24	Rumble in the Bronx (1995)	1996-02-23
25	25	Birdcage, The (1996)	1996-03-08
26	26	Brothers McMullen, The (1995)	1995-01-01
27	27	Bad Boys (1995)	1995-01-01
28	28	Apollo 13 (1995)	1995-01-01
29	29	Batman Forever (1995)	1995-01-01
30	30	Belle de jour (1967)	1967-01-01
31	31	Crimson Tide (1995)	1995-01-01
32	32	Crumb (1994)	1994-01-01
33	33	Desperado (1995)	1995-01-01
34	34	Doom Generation, The (1995)	1995-01-01

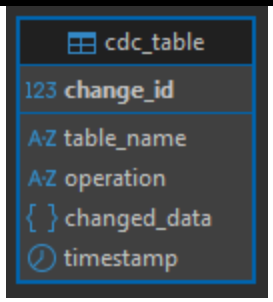
B. APARTADO B

1) Siguiendo el ejemplo que se muestra en el enlace: https://www.dbvis.com/thetable/change-data-capture-a-comprehensive-guide/ crear una tabla que llamaremos cdc_users para capturar y almacenar los cambios (borrados, añadidos y modificaciones) de la tabla users usando disparadores.

```
USE movielens;

DROP TABLE IF EXISTS cdc_table;

CREATE TABLE cdc_table (
  change_id INT AUTO_INCREMENT PRIMARY KEY,
  table_name VARCHAR(255),
  operation ENUM('INSERT', 'UPDATE', 'DELETE'),
  changed_data JSON,
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- the data capture trigger for insert

DELIMITER //

CREATE TRIGGER cdc_users_insert
AFTER INSERT ON users
FOR EACH ROW
BEGIN
  INSERT INTO cdc_table (table_name, operation, changed_data)
  VALUES (
       'users',
       'INSERT',
       JSON_OBJECT(
              'id', NEW.id,
              'age', NEW.age,
              'gender', NEW.gender,
              'occupation_id', NEW.occupation_id,
              'zip_code', NEW.zip_code
              )
       );
END //

-- the data capture trigger for update

CREATE TRIGGER cdc_users_update
AFTER UPDATE ON users
FOR EACH ROW
BEGIN
  INSERT INTO cdc_table (table_name, operation, changed_data)
  VALUES (
       'users',
       'UPDATE',
       JSON_OBJECT(
              'id', NEW.id,
              'age', NEW.age,
              'gender', NEW.gender,
              'occupation_id', NEW.occupation_id,
              'zip_code', NEW.zip_code
              )
       );
END //

-- the data capture trigger for delete

CREATE TRIGGER cdc_users_delete
AFTER DELETE ON users
FOR EACH ROW
BEGIN
  INSERT INTO cdc_table (table_name, operation, changed_data)
```

```
  VALUES (
      'users',
      'DELETE',
      JSON_OBJECT(
            'id', OLD.id,
            'age', OLD.age,
            'gender', OLD.gender,
            'occupation_id', OLD.occupation_id,
            'zip_code', OLD.zip_code
            )
      );
END //

DELIMITER ;
```

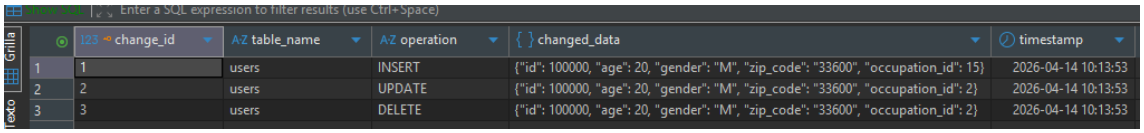


2) Posteriormente añade, borra y modifica varios registros.

   Ejecutaremos estas acciones.

```
USE movielens;

INSERT INTO users(id, age, zip_code, occupation_id, gender)
VALUES (100000, 20,'33600', 15,'M');

UPDATE users
SET occupation_id = 2
WHERE zip_code = '33600';

DELETE FROM users
WHERE zip_code = '33600';
```

3) Finalmente, realiza tres consultas sobre tabla de modificaciones cdc_users que nos muestren los registros nuevos insertados, los modificados y los borrados.



	change_id	table_name	operation	changed_data	timestamp
1	1	users	INSERT	{"id": 100000, "age": 20, "gender": "M", "zip_code": "33600", "occupation_id": 15}	2026-04-14 10:13:53
2	2	users	UPDATE	{"id": 100000, "age": 20, "gender": "M", "zip_code": "33600", "occupation_id": 2}	2026-04-14 10:13:53
3	3	users	DELETE	{"id": 100000, "age": 20, "gender": "M", "zip_code": "33600", "occupation_id": 2}	2026-04-14 10:13:53

C. APARTADO C

1) Habilitar binlog en tu servidor MySql en tu Ubuntu con WSL: En este ejercicio utilizaremos CDC basado en Binlog de MySql.

   Modificamos el fichero "/etc/mysql/mysql.conf.d/mysqld.cnf" y reiniciamos el servicio.

```
# The following can be used as easy to replay backup logs or for replication.
# note: if you are setting up a replication slave, see README.Debian about
#       other settings you may need to change.
server-id               = 27
log_bin                 = /var/log/mysql/mysql-bin.log
# binlog_expire_logs_seconds    = 2592000
max_binlog_size   = 100M
# binlog_do_db          = include_database_name
# binlog_ignore_db      = include_database_name
```

2) Inicialmente nos centraremos solamente en la captura de las inserciones sobre las tablas users y ratings:

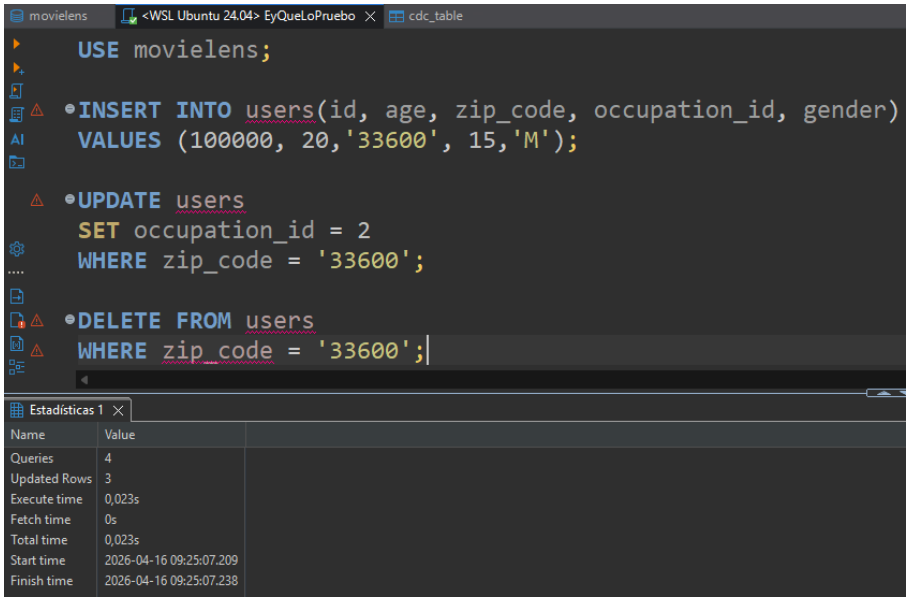
   a) *Diseña un programa en Python que capture los datos insertados y nos los almacene en un fichero llamado "nuevos.csv". Este fichero "csv" ha de tener los campos: fecha de inserción, nombre de la tabla y un objeto JSON con los campos añadidos y sus valores.*

```
from pymysqlreplication import BinLogStreamReader
from pymysqlreplication.row_event import WriteRowsEvent, UpdateRowsEvent,
DeleteRowsEvent
import csv
import json
from datetime import datetime

# Configuración de conexión
mysql_settings = {
    'host': 'localhost',
    'port': 3306,
    'user': 'usuarioremoto',
    'passwd': '123456'
}

stream = BinLogStreamReader(
    connection_settings=mysql_settings,
    server_id=27,
    only_events=[WriteRowsEvent, UpdateRowsEvent, DeleteRowsEvent],
    blocking=True,
    resume_stream=True,
    only_tables=['cdc_table'],
    only_schemas=['movielens']
)

try:
```

```
    with open('nuevos.csv', mode='a', newline='', encoding='utf-8') as file:
        writer = csv.writer(file)

        for binlogevent in stream:
            for row in binlogevent.rows:
                fecha_insercion = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")
                nombre_tabla = binlogevent.table

                datos_json = json.dumps(row["values"], default=str)

                writer.writerow([fecha_insercion, nombre_tabla, datos_json])
                file.flush()

                print(f" Registrada inserción en {nombre_tabla}:
{datos_json}")

except KeyboardInterrupt:
    print("\nPrograma detenido por el usuario.")
finally:
    stream.close()
```

b) Realiza varias inserciones de registros nuevos en ambas tablas.

Ejecutamos el script varias veces



c) Posteriormente ejecuta el programa anterior y muestra su correcta ejecución.

```
  GNU nano 7.2                                                                                    nuevos.csv
operation,"changed_data","timestamp"
INSERT,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 15}","2026-04-14 10:13:53"
UPDATE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-14 10:13:53"
DELETE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-14 10:13:53"
INSERT,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 15}","2026-04-15 11:44:47"
UPDATE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-15 11:44:47"
DELETE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-15 11:44:47"
INSERT,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 15}","2026-04-16 09:23:35"
UPDATE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 09:23:35"
DELETE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 09:23:35"
INSERT,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 15}","2026-04-16 09:24:42"
UPDATE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 09:24:42"
DELETE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 09:24:42"
INSERT,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 15}","2026-04-16 09:25:07"
UPDATE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 09:25:07"
DELETE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 09:25:07"
INSERT,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 15}","2026-04-16 09:51:12"
UPDATE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 09:51:12"
DELETE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 09:51:12"
```

d) Modifica el programa anterior de modo que lleve seguimiento de la última fecha de ejecución del programa.

```python
from pymysqlreplication import BinLogStreamReader
from pymysqlreplication.row_event import WriteRowsEvent, UpdateRowsEvent,
DeleteRowsEvent
import csv
import json
import os
from datetime import datetime

LOG_FECHA_ARCHIVO = 'ultima_ejecucion.txt'

def obtener_ultima_fecha():
    if os.path.exists(LOG_FECHA_ARCHIVO):
        with open(LOG_FECHA_ARCHIVO, 'r') as f:
            return f.read().strip()
    return "Nunca (primera ejecución)"

def guardar_fecha_actual():
    ahora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(LOG_FECHA_ARCHIVO, 'w') as f:
        f.write(ahora)

print(f"--- Última ejecución registrada: {obtener_ultima_fecha()} ---")

# Configuración de conexión
mysql_settings = {
    'host': 'localhost',
    'port': 3306,
    'user': 'usuarioremoto',
    'passwd': '123456'
}

stream = BinLogStreamReader(
    connection_settings=mysql_settings,
    server_id=27,
    only_events=[WriteRowsEvent, UpdateRowsEvent, DeleteRowsEvent],
    blocking=True,
```

```
    resume_stream=True,
    only_tables=['cdc_table'],
    only_schemas=['movielens']
)

try:
    with open('nuevos.csv', mode='a', newline='', encoding='utf-8') as file:
        writer = csv.writer(file)

        for binlogevent in stream:
            for row in binlogevent.rows:
                fecha_insercion = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")
                nombre_tabla = binlogevent.table

                # Manejo de diferentes tipos de eventos para extraer los
datos
                if isinstance(binlogevent, UpdateRowsEvent):
                    datos = row["after_values"]
                else:
                    datos = row["values"]

                datos_json = json.dumps(datos, default=str)

                writer.writerow([fecha_insercion, nombre_tabla, datos_json])
                file.flush()

                print(f" Registrada operación en {nombre_tabla}:
{datos_json}")

except KeyboardInterrupt:
    print("\nPrograma detenido por el usuario.")
finally:
    guardar_fecha_actual()
    stream.close()
```

e) *Haz las modificaciones necesarias para que en siguientes ejecuciones nos añade los nuevos registros en el "csv" solamente si son posteriores a la fecha de la última ejecución que hemos lanzado.*

```
from pymysqlreplication import BinLogStreamReader
from pymysqlreplication.row_event import WriteRowsEvent, UpdateRowsEvent,
DeleteRowsEvent
import csv
import json
import os
from datetime import datetime

LOG_FECHA_ARCHIVO = 'ultima_ejecucion.txt'
```

```
FORMATO_FECHA = "%Y-%m-%d %H:%M:%S"

def obtener_ultima_fecha():
    if os.path.exists(LOG_FECHA_ARCHIVO):
        with open(LOG_FECHA_ARCHIVO, 'r') as f:
            fecha_str = f.read().strip()
            return datetime.strptime(fecha_str, FORMATO_FECHA)
    return None

def guardar_fecha_actual():
    ahora = datetime.now().strftime(FORMATO_FECHA)
    with open(LOG_FECHA_ARCHIVO, 'w') as f:
        f.write(ahora)

ultima_ejecucion = obtener_ultima_fecha()
if ultima_ejecucion:
    print(f"--- Buscando registros posteriores a: {ultima_ejecucion} ---")
else:
    print("--- Primera ejecución: se procesarán todos los registros
disponibles ---")

mysql_settings = {'host': 'localhost', 'port': 3306, 'user': 'usuarioremoto',
'passwd': '123456'}

stream = BinLogStreamReader(
    connection_settings=mysql_settings,
    server_id=27,
    only_events=[WriteRowsEvent, UpdateRowsEvent, DeleteRowsEvent],
    blocking=False,
    resume_stream=True,
    only_tables=['cdc_table'],
    only_schemas=['movielens']
)

try:
    with open('nuevos.csv', mode='a', newline='', encoding='utf-8') as file:
        writer = csv.writer(file)

        for binlogevent in stream:
            fecha_evento = datetime.fromtimestamp(binlogevent.timestamp)

            if ultima_ejecucion and fecha_evento <= ultima_ejecucion:
                continue

            for row in binlogevent.rows:
                nombre_tabla = binlogevent.table
                datos = row["after_values"] if isinstance(binlogevent,
UpdateRowsEvent) else row["values"]
                datos_json = json.dumps(datos, default=str)
```

```
                writer.writerow([fecha_evento.strftime(FORMATO_FECHA),
nombre_tabla, datos_json])
                file.flush()

                print(f" [{fecha_evento}] Nuevo registro en {nombre_tabla}")

except KeyboardInterrupt:
    print("\nPrograma detenido.")
finally:
    guardar_fecha_actual()
    stream.close()
```

D. APARTADO D

1) Programa el servicio cron de tu Ubuntu en WSL para que todos los días entre las 9 y las 10 de la mañana nos lance el programa Python del ejercicio anterior cada 15 minutos.

- Pasamos el script a Ubuntu

```
pablo@A26P42:~/log_mysql$ ll
total 12
drwxr-xr-x 2 pablo pablo 4096 Apr 16 10:09 ./
drwxr-x--- 5 pablo pablo 4096 Apr 16 10:09 ../
-rw-r--r-- 1 pablo pablo 2194 Apr 16 10:09 guarda_log.py
```

- Ajustamos el crontab

```
  GNU nano 7.2
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0,15,30,45 9 * * * python /home/pablo/log_mysql/guarda_log.py
```

2) Realiza distintas operaciones de añadido, modificación y borrado de registros sobre las tablas users y ratings y verifica que cron lanza las tareas correctamente guardando las modificaciones en los archivos csv correspondientes.

```
pablo@A26P42:~/log_mysql$ ll
total 20
drwxr-xr-x 2 pablo pablo 4096 Apr 16 09:45 ./
drwxr-x--- 5 pablo pablo 4096 Apr 16 09:17 ../
-rw-r--r-- 1 pablo pablo   84 Apr 16 09:45 guarda_log.py
-rw-r--r-- 1 pablo pablo  801 Apr 16 09:45 nuevos.csv
-rw-r--r-- 1 pablo pablo    2 Apr 16 09:45 ultima_ejecucion.txt
```

```
  GNU nano 7.2                                                                                    nuevos.csv
operation,"changed_data","timestamp"
INSERT,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 15}","2026-04-16 10:02:48"
UPDATE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 10:02:48"
DELETE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 10:02:48"
INSERT,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 15}","2026-04-16 10:40:49"
UPDATE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 10:40:49"
DELETE,"{""id"": 100000, ""age"": 20, ""gender"": ""M"", ""zip_code"": ""33600"", ""occupation_id"": 2}","2026-04-16 10:40:49"
```